Scale Basic English

Scale Basic English is the controlled sentence language for specifications. It is deliberately smaller than English, and deliberately literal: every sentence that the executable parser accepts has exactly one reading, and every sentence it cannot read is refused by name.

This document defines the language. Decision D18 records why it exists and what it supersedes. Decision D8 remains the vocabulary rule: domain words come from the image, not from a hardcoded list in the parser.

Basic English is a starting point, not a formal grammar

Ogden's Basic English is a restricted lexicon: a core list of 850 words and a small operator set, with technical terms allowed when they are defined. It is a style and a vocabulary, not a grammar with a formal semantics. Simple English is also not a grammar; it is a readability convention used by encyclopedias and news.

Scale Basic English borrows the discipline of both, then supplies what neither supplies: a grammar, a canonical logical form, and a refusal for every construct whose meaning is not defined. The target is a child-facing specification sentence, not a general-purpose natural-language interface.

Surface forms and canonical tokens

The tokenizer maps a surface form to a canonical vocabulary token before the parser sees it. Multiword forms are one token:

```
"that is alive"   -> isAlive      (a FieldName whose form begins with "is ")
"has health"       -> has health  (a Copula followed by a FieldName)
"greater than"     -> greaterThan (a Comparison)
"can jump"         -> canJump     (a FunctionName)
```

The parser is defined over canonical tokens. A technical noun, field, or verb is added to the image vocabulary and becomes available without editing the grammar. Closed-class words are fixed by this document.

Grammar

The executable grammar is:

```
sentence    ::= statement | transition
statement   ::= quantifier subject relative? verb "."
transition  ::= "when" subject ("that")? disjunction "," "every" subject verb "."
quantifier  ::= "every" | "no" | "some"
relative    ::= "that" disjunction
disjunction ::= conjunction ("or" conjunction)*
conjunction ::= condition ("and" condition)*
condition   ::= predicate
              | "is" field
              | "has" field comparison integer
predicate   ::= a FieldName whose surface form begins with "is "
verb        ::= a FunctionName
comparison  ::= "greaterThan" | "atLeast" | "lessThan" | "atMost" | "equalTo"
integer     ::= an optionally signed base-10 integer
```

The grammar is order-sensitive. A statement's quantifier is sentence-initial, the subject is a TypeName, the relative clause is optional and introduced by `that`, and the verb phrase is final. A full stop is required. There is no implicit subject and no default verb.

A transition (D55) is the event-triggered shape: sentence-initial `when`, a TypeName subject, at least one condition, a structural comma, the universal outcome, and a full stop. The trigger and the effect must name the same subject; the grammar refuses pronouns and articles, so the repetition is explicit rather than an anaphor the parser invents. The outcome is `every` only: an effect is required, not negated or merely possible. The trigger's conditions are the precondition and the verb is the effect, so the emitted property is `precondition ==> effect` under `SentencePattern.Transition`. See `docs/decision-records.typ` D55.

`and` binds tighter than `or` (D58): a relative clause is a disjunction of conjunctions, so `a and b or c` is `(a ∧ b) ∨ c`. `and` and `or` join conditions in one relative clause; neither joins sentences. A single conjunctive clause is flattened to its atoms, so a sentence without `or` has exactly the executable shape it had before D58. The canonical text is `(a and b) or (c)`.

Quantifiers

Surface	Logical form	Executable pattern
`every`	universal affirmative	`Capability`
`no`	universal negative	`Invariant`
`some`	existential	`Possibility` (witness predicate + satisfiability, D33)
`all`	not admitted	`UnsupportedQuantifier "all"`; use `every`

`no` is not an ordinary quantifier that the emitter happens to read as a `Capability`. It is a universal negative, and the emitter renders it as a negated postcondition under the same preconditions.

`some` is existential, not universal: the emitter renders a **witness predicate** (`pre && post`) and the SMT obligation is `pre ∧ post` satisfiability, answering `Witnessed model` or `NoWitness` (D33). Reading `some` as `every` would be a silent change of meaning. `all` is not part of the controlled surface; it is refused with a direct instruction to use `every`.

A transition's outcome admits only `every`; `no` and `some` are refused with `TransitionOutcomeNotUniversal`.

Conditions

A condition is one of:

- a boolean predicate phrase, such as `is alive`, canonicalized to `isAlive`;
- an explicit copula and field, `is <field>`;
- a numeric comparison, `has <field> <comparison> <integer>`.

The executable projection supports all three forms. A numeric comparison is carried as `Types.NumericCondition(field, comparison, value)`, so the grammar, the emitter, the SMT obligation, and the repair gate share one typed condition (D24). The emitted record widens a numeric field to `float` and the predicate compares against a float literal (`health > 0.0`); the SMT theory stays integer-valued, so a non-integral threshold is `Unavailable`, never rounded.

A bare field noun is not a sentence condition. `Every player that alive can jump.` is refused; the surface form must be a predicate phrase or an explicit copula. `Every player that health can jump.` is refused for the same reason.

Numeric comparisons are integer comparisons. The SMT theory is integer-valued, so a decimal literal is refused rather than rounded. The comparisons are `greater than`, `at least`, `less than`, `at most`, and `equal to`.

Closed-class words

The following are deliberately outside the current surface:

Word or form	Refusal	Reason
`the`, `a`	`ArticleNotAllowed`	no articles in a universal specification
pronouns such as `they`	`PronounNotAllowed` or `UnknownToken`	no anaphora or antecedent resolution
`when` away from the sentence start	`ConditionalNotAllowed`	only a transition licenses `when`, and only sentence-initially
`not`, `never`	`NegationNotAllowed`	use sentence-initial `no`
`all`	`UnsupportedQuantifier`	use `every`; the subject stays singular

`or` is admitted between conditions (D58). The parser does not guess. An unknown word, an inactive word, a missing full stop, a misplaced token, a missing comparison, a missing transition comma, and a mismatched transition subject each have a named `ParseError` case.

Canonical logical form

A sentence means:

```
<quantifier> <subject> : <condition-list> -> <verb>

condition-list ::= conjunction ("or" conjunction)*
conjunction    ::= condition ("and" condition)*
```

with the quantifier read as follows:

- `every`: for every subject that satisfies the conditions, the verb holds;
- `no`: for every subject that satisfies the conditions, the verb does not hold;
- `some`: there exists a subject that satisfies the conditions and the verb (D33).

The conditions are a disjunction of conjunctions. `and` binds tighter than `or`, so `a and b or c` is `(a and b) or (c)`. There is no implication inside a condition and no nested relative clause. A specification that needs those constructs is refused until the language defines them.

Examples

Accepted by the executable parser:

```
Every player that is alive can jump.
Every player can run.
No player that is alive can jump.
No player can jump.
Some player that is alive can jump.
Every player that is alive and has health greater than 0 can jump.
Every player that is alive or has health greater than 0 can jump.
Every player that is alive and has health greater than 0 or has health less than 0
can jump.
When player is alive, every player can jump.
When player is alive and has health greater than 0, every player can jump.
When player is alive or has health greater than 0, every player can jump.
```

Refused by the grammar:

```
The player never can run.                                    -> ArticleNotAllowed "the"
Never can jump.                                              -> NegationNotAllowed
"never"
When a player is alive they can jump.                         -> ArticleNotAllowed "a"
player can jump.                                             -> ExpectedQuantifier
"player"
can jump every player.                                       -> ExpectedQuantifier
"canJump"
Every player can jump                                        -> MissingFullStop
Every player that is alive and can jump.                     -> ExpectedCondition
"canJump"
Every player that is alive or health can jump.               -> ExpectedCondition
"health"
Every player that has health greater than zero can jump.     -> NotAnInteger "zero"
Every player that alive can jump.                            -> ExpectedCondition
"alive"
All player can run.                                          -> UnsupportedQuantifier
"all"
When player is alive, no player can jump.                    ->
TransitionOutcomeNotUniversal "no"
When player is alive, every enemy can jump.                  ->
TransitionSubjectMismatch("player", "enemy")
When player is alive every player can jump.                  -> ExpectedTransitionComma
"every"
```

Why this shape

The grammar is strict because the pipeline is a specification tool, not a chatbot. A sentence that is accepted becomes an emitted property and an SMT obligation. If the parser guessed at an article, a pronoun, a disjunction, or an existential, the proof would be about a sentence the author did not write.

The grammar is also small because the project's goal is a literal language, not natural-language coverage. Technical vocabulary is open, but the sentence structure is closed. That is the Basic English discipline: a small operator set, defined terms, and no rhetorical ambiguity.

Staged completion

Stage 1, recorded as D18 and pinned by the suite:

- strict quantifier, subject, relative clause, condition, verb, and full stop;
- `every` is `Capability`, `no` is `Invariant`, and `some` is `Possibility` (D33: witness predicate plus satisfiability);
- named refusals for articles, pronouns, `when` away from a transition's start, negation, `all`, unknown words, and malformed sentences;
- a structural `has` condition in the grammar and in the executable projection (D24);
- one Hedgehog property that generates valid sentences, including numeric `has`, and pins their canonical form.

Stage 2, recorded as D55:

- the event-triggered shape `when <subject> <condition>, every <subject> <verb>.` projects to `Transition`;
- the trigger's conditions are the precondition and the verb is the effect, so the F# artifact is a guarded effect and the SMT obligation is universal;
- the trigger and effect must name the same subject, the outcome is universal, and the comma is structural; each failure has a named `ParseError`;

- the state-change rendering (before/after, or a transition function) remains deferred because the specification and the verification lattice carry one subject, not a pair of states.

Stage 3, recorded as D58:

- `and` binds tighter than `or`; a relative clause (and a transition trigger) is a disjunction of conjunctions;
- the executable shape is one `Condition.Disjunction` atom over atomic conjunctions, flattened to its atoms when there is no `or`, so every earlier sentence and pinned artifact is unchanged;
- the F# precondition is `||` over parenthesized `&&` clauses, the SMT obligation is `MkOr` over `MkAnd`, and the repair gate evaluates the DNF;
- the canonical `(a and b) or (c)` text round-trips without a corpus-schema change.

No named sentence-logic stage remains. Nested parentheses beyond the canonical DNF text, `not`, and implication are still outside the surface and are refused by name.